mendix-test-agent

Record once. AI-enrich. Regress forever.

An AI-assisted end-to-end testing platform: record a manual test pass once, let an LLM turn it into a maintainable, self-healing automated test, review it like a code change, then run it forever as part of continuous regression.

Project report

Generated 2026-09-14

Reference implementation target: Mendix low-code applications

1. Executive Summary

mendix-test-agent is a browser-testing platform built around one idea: the expensive, error-prone part of end-to-end testing isn't clicking through a flow once — it's turning that flow into a test that stays correct as the application under test keeps changing. This project automates exactly that transition, and then keeps the resulting test alive automatically instead of letting it quietly rot.

A tester records a flow the same way they'd manually test it — logging in, clicking through a scenario, no code. An LLM (Groq, OpenAI-compatible) then rewrites that raw recording into a real Playwright test: stable locators, explicit assertions inferred from what actually happened, proper asynchronous waits instead of fixed sleeps, and BDD-style step labels. Every AI-generated change — a brand-new test, a re-recording, or a proposed fix to a broken locator — goes through a mandatory human review queue before it ever reaches the real test suite. Nothing the AI writes is trusted blindly.

Beyond generation, the platform actively maintains its own tests: it detects when a locator breaks and proposes a grounded fix (never an invented one — always checked against a live replay of the real application), diagnoses failures in plain English, scores every test's maintainability, flags flaky tests from real run history, and — a genuinely novel piece — can reach directly into the application's own database to clear stuck sessions and diagnose authorization gaps between user roles.

> **In one sentence:** it turns "someone tested this once by hand" into "this is permanently, automatically re-tested, and the tests fix themselves when the application's UI changes underneath them."

2. Problem & Value Proposition

The problem this solves

- **Manual regression testing doesn't scale.** Every UI change risks breaking something a human tested once and never re-checks.
- **Hand-written E2E tests are expensive and brittle.** Writing a Playwright test by hand for every flow is slow; the locators it depends on break the moment a widget gets renamed, and someone has to notice, diagnose, and fix it.
- **Low-code platforms change fast and can't be inspected programmatically.** There's no access to a Mendix Studio Pro project's internal model from outside it — the only ground truth available to an external tool is what a real browser session actually renders.
- **Trial/dev licenses impose real operational constraints** (session/seat limits) that most testing tools ignore entirely, and that silently produce false failures if not actively managed.

Value added

Without this project	With this project
A developer or QA hand-writes each Playwright test, guessing at locators	A tester records a flow once by clicking through it; the test is generated from real, replayed DOM state
A renamed widget silently breaks a test; someone has to notice, open the test, guess the fix	The broken locator is detected, a fix is proposed from what's genuinely on the page right now, and a human just approves or rejects it
A failed test shows a raw stack trace; diagnosing it costs time	Every failure gets a plain-English "what happened / likely cause / recommended action" diagnosis alongside the real error
Test quality/maintainability is subjective, unmeasured	Every test gets a deterministic, inspectable quality score (locator stability, assertion coverage, async handling, BDD structure)
"Is role X blocked from role Y's data?" is answered by manual poking	An automated boundary scan compares what each role's browser session can actually see, flags real exposure
A stuck session/seat-limit issue means restarting the whole app	A direct, targeted database-level session reset — seconds, not an app restart

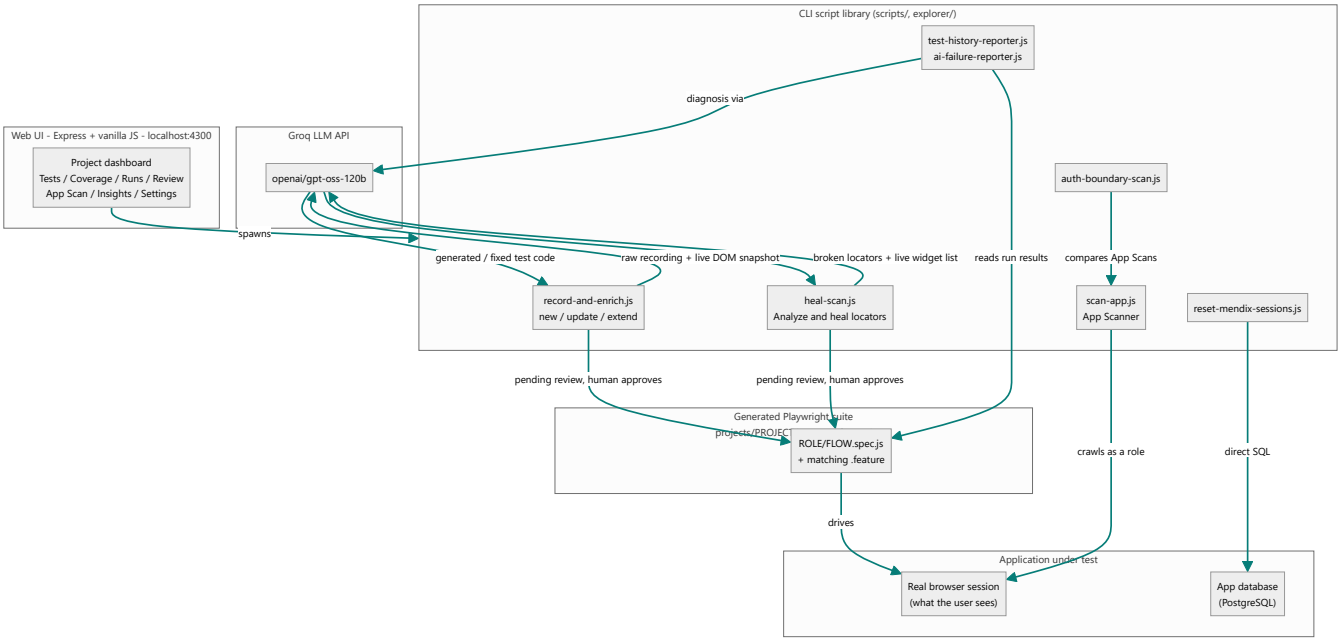
3. Technology Stack

Layer	Technology	Role in this project
Browser automation	**Playwright** (+ Playwright Test runner)	Records flows (codegen), drives every generated test, powers live-replay verification for locator healing and quality checks
Runtime / language	**Node.js** (native ESM), plain **JavaScript**	Every script, the UI server, and every generated test — one language across the whole stack
AI / LLM	**Groq API** (OpenAI-compatible SDK, model `openai/gpt-oss-120b`)	Recording → test rewrite, failure diagnosis, locator-fix suggestion, assertion suggestions, natural-language-to-test-plan generation
Web UI	**Express** + a hand-rolled vanilla JS/HTML/CSS front end (no framework)	Project dashboard, live job streaming (SSE), review/approve workflow, Insights, Settings
Application database access	**PostgreSQL** (via `pg`), read directly for diagnostics and session management	Live session count/clearing, cross-role authorization comparison, one-off data-migration tooling (HSQLDB → Postgres, built live during this project)
Containers	**Podman**	Stood up a local PostgreSQL instance as an alternative to Mendix's built-in, single-process HSQLDB — needed because HSQLDB can't be reached by any external tool while the app is running
Accessibility	**@axe-core/playwright**	Opt-in WCAG 2.0/2.1 A+AA scanning of any page state
CI/CD	**GitHub Actions**	Runs the full suite on every push/PR/nightly, validates generated tests for leaked secrets, publishes HTML + JUnit reports, posts failures to Slack
Reporting	Playwright's own **HTML reporter** + **JUnit XML**	Per-run reports with a navigable BDD step tree; JUnit for external dashboard/tracker ingestion
Code quality	**ESLint**, **Prettier**, **Husky** + **lint-staged**	Pre-commit checks, including a hardcoded-secret scan on every generated test

> The stack is deliberately minimal — no test framework beyond Playwright itself, no frontend framework, no ORM. Every moving part is something the project directly controls and can reason about, which matters for a tool whose entire premise is "never guess, always verify against something real."

4. System Architecture

The system has four cooperating layers: a thin web UI, a library of CLI scripts that do the actual work (and that the UI simply orchestrates), the generated Playwright test suite itself, and the application under test — reached two ways: through the browser, like a real user, and, for maintenance operations, directly against its database.

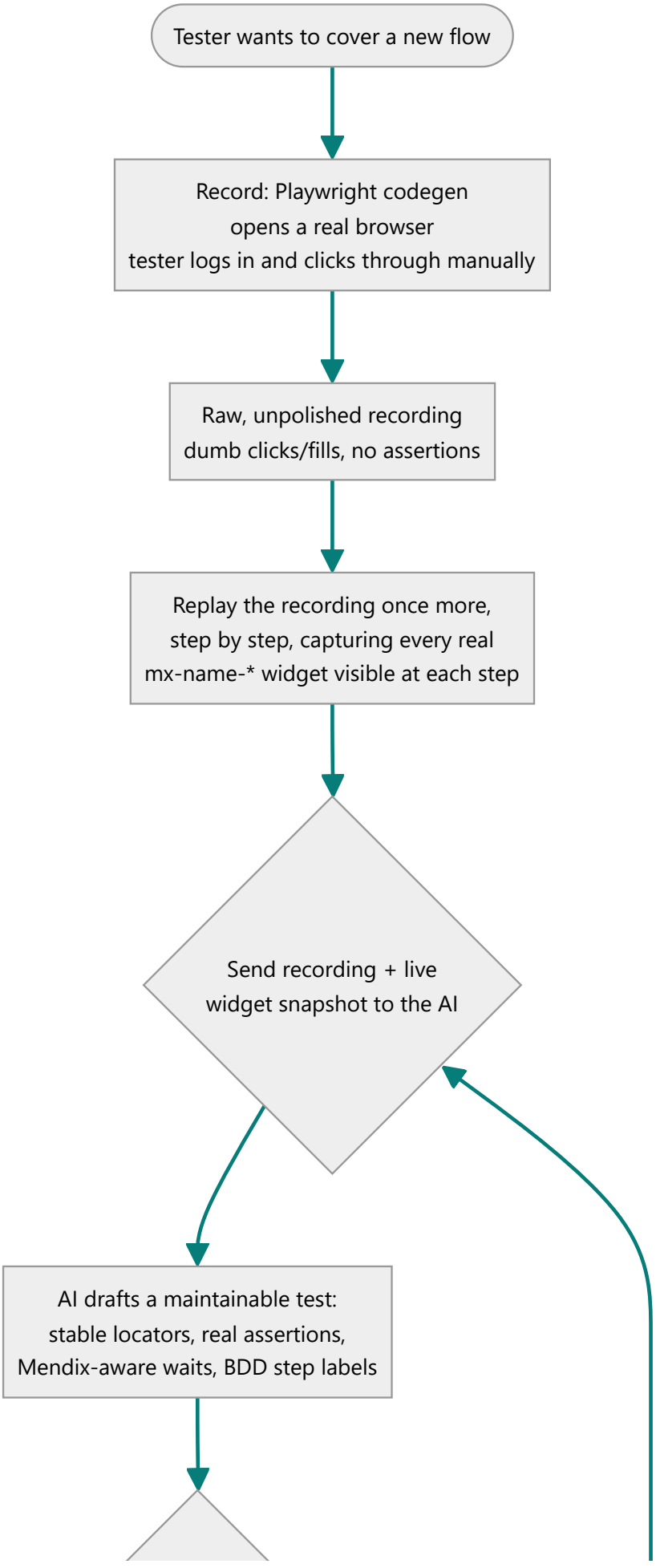


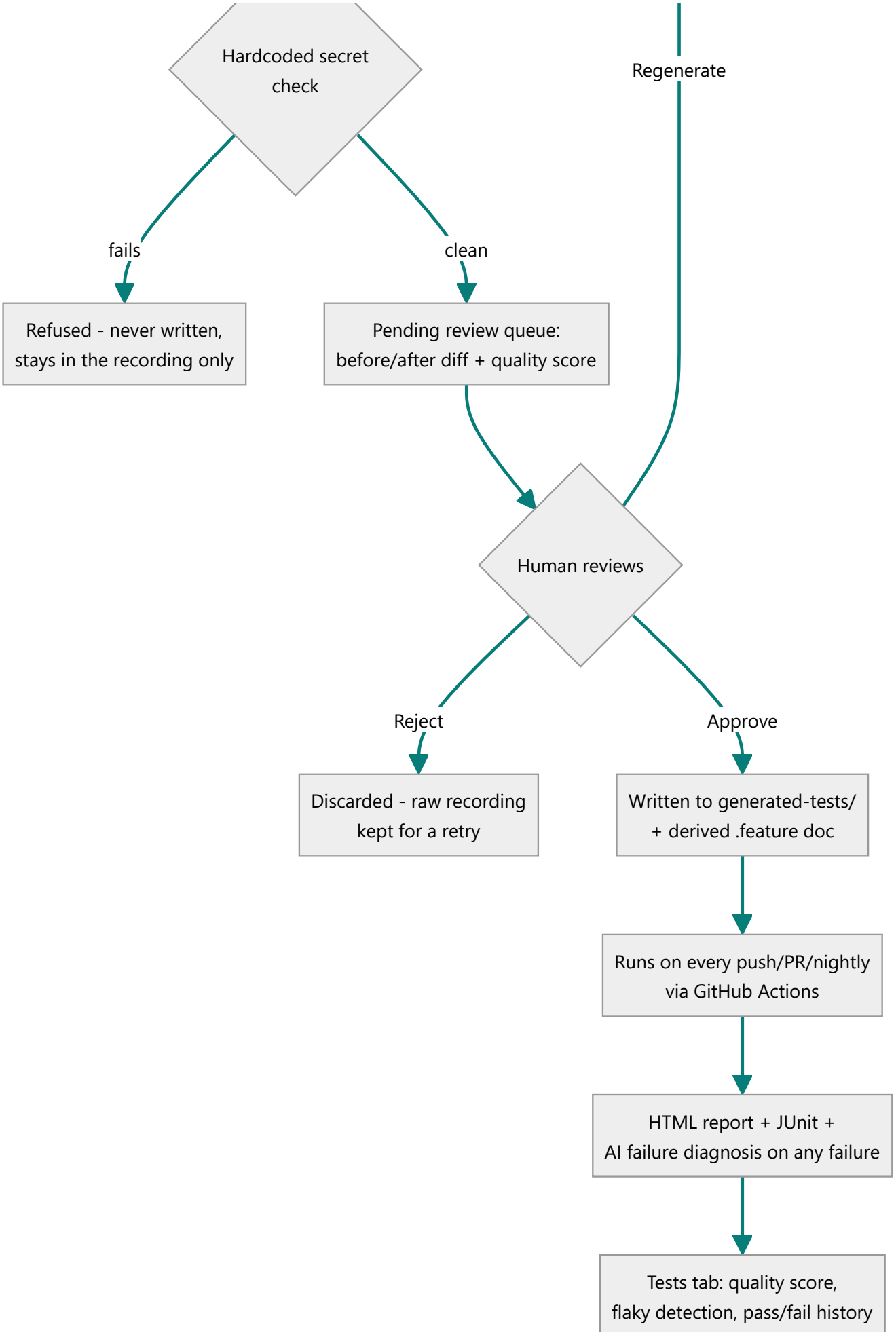
Key architectural decisions

- **The UI is a thin wrapper, never the source of truth.** Every button runs the same CLI command a person could type directly — nothing about the underlying scripts changes based on whether they're invoked from a browser or a terminal, which is also why CI uses the CLI path exclusively.
- **Nothing AI-generated is trusted automatically.** A brand-new test, a re-recording, or a locator fix all land in a pending-review queue as a before/after diff. A hardcoded-secret check runs before a proposal can even be approved.
- **Every locator decision is grounded in a live replay, never a guess.** Whether generating a new test or fixing a broken one, the model only ever sees — and is only allowed to choose from — widgets that a real, just-now browser session actually rendered.
- **Multi-project, multi-role, multi-environment by design.** The same installation can test several separate applications, each with its own roles and credentials, each runnable against several named environments (Default, QA, Staging...) with zero test-source changes.

5. Core Workflow — Record → Enrich → Review → Run

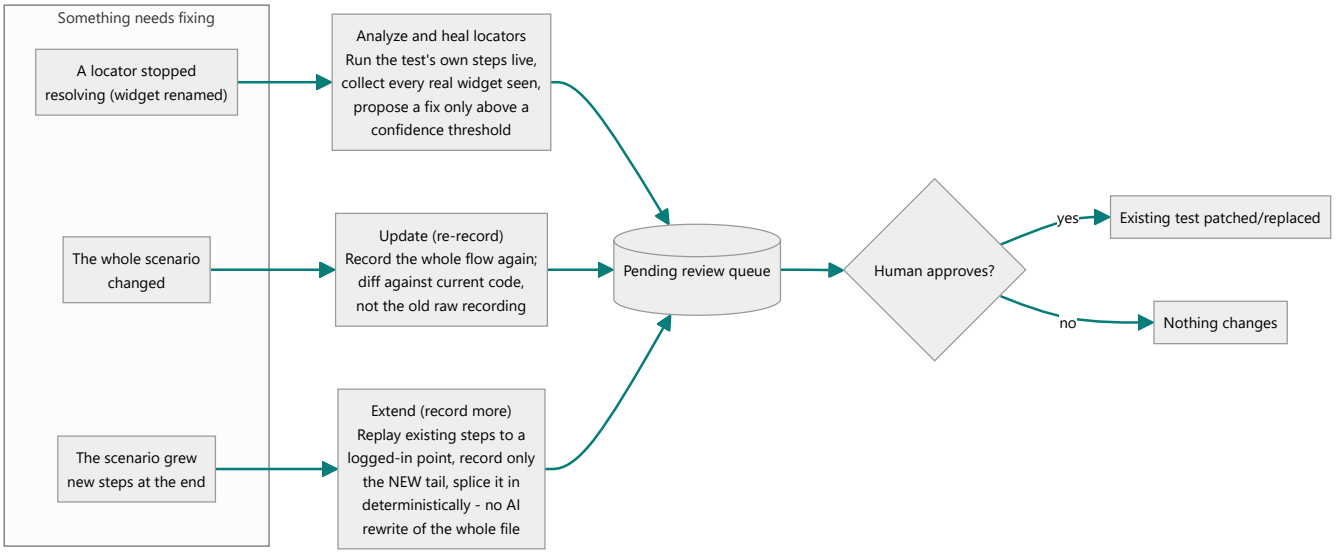
This is the primary path from "someone tested a flow by hand" to "this is a permanent, self-maintaining automated test."


Tester wants to cover a new flow
Record: Playwright codegen
opens a real browser
tester logs in and clicks through manually
Raw, unpolished recording
dumb clicks/fills, no assertions
Replay the recording once more,
step by step, capturing every real
mx-name-* widget visible at each step
Send recording + live
widget snapshot to the AI
AI drafts a maintainable test:
stable locators, real assertions,
Mendix-aware waits, BDD step labels


Hardcoded secret check
fails
clean
Refused - never written, stays in the recording only
Pending review queue: before/after diff + quality score
Regenerate
Human reviews
Reject
Approve
Discarded - raw recording kept for a retry
Written to generated-tests/ + derived .feature doc
Runs on every push/PR/nightly via GitHub Actions
HTML report + JUnit + AI failure diagnosis on any failure
Tests tab: quality score, flaky detection, pass/fail history

Self-healing sub-workflow: Update / Analyze / Extend

Once a test exists, it doesn't just sit there waiting to break. Three grounded maintenance paths cover everything that can go wrong with an existing test, all converging on the same review queue:



> The **Extend** path is a deliberate design choice worth calling out: instead of asking the AI to reproduce an entire file (risking a subtle corruption of code that was already passing), only the new tail is AI-generated, and it's spliced into the existing file by plain, deterministic text manipulation — the model never touches the parts of the file that already worked.

6. How the AI Generation Actually Works



The same "replay first, ask the model to choose only from what's real" pattern is reused, unmodified in spirit, for every AI-touched feature in the project: new-test generation, locator healing, and the natural-language-to-test-plan generator. None of them ever let the model invent a selector from nothing.

7. Feature Catalog

Generation & maintenance

- Record → AI-enrich → validate, one command
- Pending-review queue for every AI change
- Update / Analyze & heal / Extend
- Deterministic quality score per test
- Negative-path coverage heuristic
- BDD `test.step()` labels, auto-derived `.feature` docs

Diagnostics & intelligence

- Plain-English AI failure analysis on every failure
- Flaky-test detection from real run history
- Performance trend tracking (avg/P95/latest)
- Regression intelligence (diff last two runs)
- Coverage map (App Scanner vs. what tests reference)
- Test dependency graph

Exploration & security

- Mendix App Scanner — crawl reachable pages/widgets as a role
- Authorization boundary scan — cross-role exposure detection
- Autonomous LLM-driven exploration (bug-finding, not CI)
- Opt-in accessibility scanning (axe-core, WCAG A/AA)

Operations & infrastructure

- Multi-project, multi-role, multi-environment
- Direct database session reset (targeted or global)
- Automatic session clearing before/after every test
- Opt-in cross-browser coverage (Chromium/Firefox/WebKit)
- Opt-in visual regression (screenshot diffing)
- Full CI pipeline: secrets validation, reports, Slack alerts

Multi-project **Multi-role** **Multi-environment** **Self-healing** **AI diagnosis** **Quality scoring** **Flaky detection** **Accessibility** **Visual regression** **Authz scanning** **DB-level tooling** **CI/CD ready**

8. How To Use It (Brief)

1. **Setup:** `npm install`, `npx playwright install --with-deps chromium`, fill in a Groq API key and the app's URL/credentials in `.env`.
2. **Open the UI:** `npm run ui` → `http://localhost:4300`, pick or create a project.
3. **Record a flow:** Review tab → Record → log in as a role and click through the scenario like a real manual test → close the recorder window.
4. **Review the AI's draft:** a before/after diff with a quality score appears in Pending Review → Approve, Reject, or Regenerate.
5. **Run it:** Runs tab (one role, one file, by tag, or everything) — or let CI run it on every push automatically.
6. **When something breaks:** Fix an existing test → Analyze & heal locators (renamed widget), Update (scenario changed), or Extend (scenario grew) — review and approve the proposed fix like anything else.
7. **Keep an eye on health:** Tests tab for quality/flaky trends, Insights tab for performance/regression/coverage/authorization.

> The CLI equivalent of every UI action is documented in `TESTING_GUIDE.md` — the UI is optional, never the only way to do something.

9. Is This Mendix-Only?

Honest answer: **the architecture is general-purpose; the current implementation is Mendix-coupled in a specific, identifiable layer.** The two need to be judged separately.

What's already generic (reusable for any Playwright-drivable app)

- The whole record → AI-enrich → human-review → run pipeline and its orchestration.
- Multi-project / multi-role / multi-environment project structure.
- The pending-review queue, quality scoring, flaky detection, AI failure diagnosis.
- The Update / Analyze / Extend self-healing pattern — the mechanism ("replay live, only propose from what's real, splice deterministically") is not Mendix-specific at all.
- CI wiring, secret-leak validation, reporting, the whole UI shell.

What's currently Mendix-specific

Component	Why it's Mendix-coupled today
Locator strategy (`BasePage.mx()`)	Relies on Mendix's own `mx-name-<WidgetName>` CSS class convention — effectively Mendix's built-in `data-testid` equivalent
AI system prompts (`enrich.js` , `heal-scan.js`)	Written with explicit Mendix knowledge: the `mx-name-*` convention, `waitForMendixIdle()` , the `/xas/` API pattern
Session/seat-cap handling	Specific to how a Mendix trial license enforces concurrent-session limits
Database-level tooling (session reset, entity-mapping utilities)	Hardcoded against Mendix's own internal schema conventions (`system$session` , `system$user` , `mendixsystem$*`)

What it would take to target a different stack

For a React/Vue/Angular/plain-HTML application with a stable locator convention (`data-testid` , `aria-label` , or similar), porting this platform would mean:

1. Swap `BasePage` 's locator helper for whatever stable attribute the target framework guarantees.
2. Rewrite the AI system prompts to reference that convention instead of `mx-name-*` , and remove Mendix-specific async-wait guidance.
3. Drop (or reimplement against the target's own persistence layer) the database-level session/authorization tooling — those are inherently platform-specific by nature, not just by current implementation.

> **Bottom line:** a team adopting this for a non-Mendix application keeps roughly the entire orchestration, review, AI-diagnosis, and CI layer as-is, and needs to rebuild only the locator-and-prompt adapter layer plus any platform-specific database utilities. It is not a Mendix-only tool in principle — it is a Mendix-first implementation of a general pattern.

10. Known Limitations & Constraints

- **Trial license seat cap** (local/dev Mendix instances): concurrent-session limits can cause false failures under load; mitigated with serial execution, retries, and automatic session clearing, but not eliminated on a genuinely tiny trial cap.
- **No access to the Studio Pro project model.** Every feature is grounded in what a real browser session (or, for DB tooling, the app's own database) can observe — by design, but it means nothing here can reason about microflows/entities it hasn't actually seen exercised.
- **Self-healing requires a confident match.** A locator with no plausible replacement among currently-visible widgets is reported as unresolved, never guessed — correct behavior, but it means some fixes still need a human.
- **Extend mode is single-scenario only.** A spec with multiple `test()` calls in one `describe` block isn't a supported target yet.
- **Data cleanup is opt-in and manual** where no verified delete flow has been recorded — the project deliberately refuses to fabricate a delete step from a guessed locator.

11. Conclusion

mendix-test-agent demonstrates that the most expensive part of end-to-end testing — keeping tests alive as the application changes — can be substantially automated without sacrificing trust, as long as every automated decision stays grounded in something real and every AI-authored change passes through a human before it counts. The result is a testing practice that costs a single manual pass per flow instead of a manual pass every time someone wants to be sure a flow still works.

Report generated from the project's own source and documentation (`README.md` , `TESTING_GUIDE.md` , `CLAUDE.md`) as of 2026-09-14.